

National University of Computer and Emerging Sciences

Islamabad Campus

CS-1004 Object Oriented Programming Spring 2026

Assignment 1 — Sections (All)

Submission Deadline: February 15, 2026 - Sunday

Instructions

1. Assignments are to be done individually. You must complete this assignment by yourself. You cannot work with anyone else in the class or with someone outside of the class. The code you write must be your own and you must understand each part of your code. You are encouraged to get help from the instructional staff through google classroom.
2. Do not use any String or math libraries (such as cmath, cstring, string etc) and also do not use built-in functions (such as strlen, strcmp etc). Caution: zero marks will be awarded. Do not use any external library unless mentioned otherwise.
3. Do not edit Function Prototypes. Caution: zero marks will be awarded.
4. The usage of string is strictly prohibited.
5. Your code must be generic.
6. Marks distribution and test Cases are provided for each question. Your code will be evaluated with similar test cases. If the required output is generated, you will be awarded full marks. Failing to generate the correct output will result in zero marks. Total Marks: 200.
7. For PrintPattern questions, the output should be properly displayed and well presented.
8. Static and global variables are not allowed.
9. Plagiarism: Plagiarism of any kind (copying from others, copying from the internet, etc) is not allowed. If found plagiarized, you will be awarded zero marks in the whole assignment category.
10. Please start early otherwise you will struggle with the assignment.
11. Test cases: Test cases (in gtest) will be shared with you on Google Classroom. We will be running your code against our test cases, and a test case failure or a segmentation fault/incorrect result or even syntax error will result in ZERO marks.
12. Submission Guidelines: Dear students, we will be using auto-grading tools (gtest), so failure to submit according to the below format would result in ZERO marks in the relevant evaluation instrument.
 - a. Make your own file named submission.h. Please don't include the main function while submitting the file. And don't remove test cases (in testcases.cpp) or function prototypes (in submission.h).
 - b. Your submission.cpp file must contain your name, student-id, and assignment # on the top of the file in the comments.
 - c. Move your submission.h in one folder. The folder must contain only the submission.h file (no binaries, no exe files etc.). If we are unable to download your submission due to any reason you will be awarded zero marks.
 - d. Run and test your program on a lab machine before submission. If there is a syntax error, ZERO marks will be awarded in that specific question.

National University of Computer and Emerging Sciences

Islamabad Campus

- e. Rename the folder as ROLL-NUM_SECTION (e.g. 24I-0001_A) and compress the folder as a zip file. (e.g. 24I-0001_A.zip). Only zip files will be acceptable.
- f. Submit the .zip file on Google Classroom within the deadline.
- g. Submissions other than Google classroom (e.g. email etc.) will not be accepted.
- h. The student is solely responsible to check the final zip files for issues like corrupt files, viruses in the file, mistakenly exe sent. If we cannot download the file from Google classroom due to any reason it will lead to zero marks in the assignment. You are required to use Visual Studio 19 or above for the assignment.
- i. For the purpose of submission, a google form will be uploaded. No submission on GCR submission portal will be accepted.

Note: Follow the given instruction to the letter, failing to do so will result in a zero.

Shapeshifter [25 marks]

You are given a raw flattened array of integers. Your task is to implement a function that generates all possible valid 3D **views** of this flattened array.

The Prototype

```
int*** CreateViews(int* data, int size, int& count);
```

The 'int***' returned by the function defines the view where each pointer points to a single view of all possible views (hence a triple pointer).

You **MUST NOT COPY** the data into newly allocated space; you only need to maintain pointers to the original 1D array.

INPUTS:

- A pointer to the 1D array containing the ints
- The size of the 1D array
- A variable count passed as reference that you must populate with the number of possible views.

OUTPUT:

- A dynamic array (int***) containing int** items

Constraints

- You are not allowed to use the '[']' operator. You must leverage pointer arithmetic.
-

Memory Archivist [50 marks]

I. Scenario

The year is 2077. A decommissioned autonomous archive has been recovered from a collapsed data vault. Inside lies a single corrupted **Memory Tape** (a null-terminated character sequence).

Your Mission: Reconstruct the Memory Archivist system—a powerful string manipulation engine that operates entirely through pointer arithmetic and includes a complete version control system to track and undo changes.

II. The "Iron Rules" (Strict Constraints)

The Pointer-Only Mandate:

- You are **STRICTLY FORBIDDEN** from using the array indexing operator `[]`
- ALL traversal must be done using pointer arithmetic (e.g., `while(ptr < end), ptr++, *ptr`)

No Standard String Libraries:

- You cannot use `<cstring>`, `<string>`, or functions like `strlen`, `strcpy`, `strcmp`
- No STL containers allowed, except `<fstream>` for file I/O and `<string>` for `getline` and other file related functions. **STRINGS CANNOT BE USED FOR IMPLEMENTATIONS OF FUNCTIONS.**

Manual Memory Management:

- Use only `new`, `new[]`, `delete`, and `delete[]`
- No memory leaks allowed

PART 1: The String Engine (20 Functions)

Implement the following 20 functions using **only pointer arithmetic**.

A. Easy Operations (4 Functions)

1. `int getLength(const char* str)`

Calculate string length using pointer traversal.

National University of Computer and Emerging Sciences

Islamabad Campus

2. **char* copyString(const char* str)**

Create a deep copy of the string in new memory.

3. **bool areEqual(const char* str1, const char* str2)**

Compare two strings for equality.

4. **char* concatenate(const char* str1, const char* str2)**

Merge two strings into a new memory allocation.

B. Intermediate Operations (8 Functions)

5. **void toUpperCase(char* str)**

Convert lowercase to uppercase in-place.

6. **void toLowerCase(char* str)**

Convert uppercase to lowercase in-place.

7. **int findFirstOccurrence(const char* str, char target)**

Return the offset of the first occurrence of **target** (return -1 if not found).

8. **int findSubstring(const char* str, const char* substring)**

Return the offset of the first occurrence of **substring** (return -1 if not found).

9. **char* reverseString(const char* str)**

Return a new string that is the reverse of the input.

10. **char* removeCharacter(const char* str, char target)**

Return a new string with all instances of **target** removed.

11. **char* substring(const char* str, int start, int length)**

Extract a specific portion of the string into a new allocation.

12. **char* replaceChar(const char* str, char oldChar, char newChar)**

Return a new string with simple character replacement.

C. Hard Operations (6 Functions)

13. **char* replaceSubstring(const char* str, const char* target, const char* replacement)**

Replace ALL occurrences of **target** with **replacement**.

14. **char* insertAt(const char* str, const char* toInsert, int position)**

Insert a substring at a specific position.

National University of Computer and Emerging Sciences

Islamabad Campus

15. `char* deleteRange(const char* str, int start, int length)`

Delete a specific range of characters and resize.

16. `char* compress(const char* str)`

Run-Length Encoding (e.g., "aaabbc" → "a3b2c1").

17. `char* decompress(const char* str)`

Expand RLE strings back to original form.

18. `char* rotateString(const char* str, int positions)`

Rotate characters left (positive positions) or right (negative).

PART 2: Version Control System

You will implement a **two-layer version control system**:

- **Layer 1 (Memory)**: Track mutations in a dynamically allocated array
- **Layer 2 (File)**: Persist mutations to a log file for permanent storage

LAYER 1: In-Memory Mutation Tracking

You must maintain **three global variables** that grow dynamically:

- `char** mutationHistory;` // Stores copies of the tape after each mutation
- `int mutationCapacity;` // Current capacity of arrays
- `int mutationCount;` // Number of mutations stored

How it works:

- Every time you modify the tape, create a **copy** of the new state
- Store this copy in `mutationHistory` at index `mutationCount`
- Increment `mutationCount`
- If `mutationCount` reaches `mutationCapacity`, **double the capacity** and resize arrays

Initial Setup:

- `mutationHistory = new char*[10];` // Start with capacity 10

National University of Computer and Emerging Sciences

Islamabad Campus

- `mutationCapacity = 10;`
- `mutationCount = 0;`

Example:

- Index 0: "hello" (original)
- Index 1: "HELLO" (after `toUpperCase`)
- Index 2: "HELLO WORLD" (after `concatenate`)
- Index 3: "OLLEH DLROW" (after `reverse`)

To ensure your code works with the test cases, you **MUST** implement the following two helper functions in your submission file:

void initializeVersionControl(int size)

Initialize the mutation state managers.

void cleanupVersionControl()

Proper memory de-allocation of the mutation state managers.

LAYER 2: Persistent File Logging

Every mutation must also be logged to a file: `mutation_log.txt`

File Format (one line per mutation):

- `[INDEX]|[OPERATION]|[INVERSE_OPERATION]|[PARAMETERS]`

Why store inverse operations?

To undo a mutation, we don't need the old string—we just apply the *inverse* of what we did!

Examples:

Original Operation

`toUpperCase()`

Log Entry

`1|UPPERCASE|LOWERCASE|`

National University of Computer and Emerging Sciences

Islamabad Campus

<code>rotateString(str, 3)</code>	<code>2 ROTATE ROTATE -3</code>
<code>insertAt(str, "abc", 5)</code>	<code>3 INSERT DELETE 5,3</code>
<code>replaceChar(str, 'a', 'b')</code>	<code>4 REPLACE_CHAR REPLACE_CHAR b,a</code>

Key Points:

- Inverse of `toUpperCase` is `toLowerCase`
- Inverse of `rotate(+3)` is `rotate(-3)`
- Inverse of `insert("abc", 5)` is `delete(5, 3)` (delete 3 chars at position 5)
- Inverse of `replaceChar('a','b')` is `replaceChar('b','a')`

19. `void recordMutation(char* newTapeState, const char* operation, const char* inverseOp, const char* params)`

Records a mutation to both memory and file.

Requirements:

- Create a deep copy of `newTapeState`
- Add to `mutationHistory[mutationCount]`
- If capacity is full, double the array size using pointer arithmetic
- Append to `mutation_log.txt` in the specified format
- Increment `mutationCount`

Example usage:

- `char* tape = copyString("hello");`
- `tape = toUpperCase(tape);` // tape is now "HELLO"
- `recordMutation(tape, "UPPERCASE", "LOWERCASE", "");`

20. `void displayHistory()`

Display the entire mutation history.

Requirements:

- Read and display contents of `mutation_log.txt`

National University of Computer and Emerging Sciences

Islamabad Campus

- Show each mutation in a readable format
- Use pointer arithmetic to traverse and print

Example output:

- === MUTATION HISTORY ===
- [0] Original State
- [1] UPPERCASE (inverse: LOWERCASE)
- [2] ROTATE +3 (inverse: ROTATE -3)
- [3] INSERT at position 5 (inverse: DELETE at 5, length 3)

21. `char* undoLastMutation()`

Undo the most recent mutation.

Requirements:

- Return a deep copy of `mutationHistory[mutationCount - 1]`
- Decrement `mutationCount`
- Update the log file to mark this mutation as undone (append `|UNDONE` to the line)

Example:

```
char* tape = undoLastMutation(); // Returns state before last change
```

22. `char* revertToVersion(int index)`

Restore the tape to a specific version from history.

Requirements:

- If `index < 0` or `index >= mutationCount`, return `nullptr`
- Return a deep copy of `mutationHistory[index]`
- Set `mutationCount = index + 1` (discard all versions after this)
- Update log file to mark discarded mutations as `|REVERTED`

Example:

- `char* tape = revertToVersion(2);` // Go back to mutation #2
- // All mutations after index 2 are now discarded

National University of Computer and Emerging Sciences

Islamabad Campus

23. `void permanentlyDeleteHistory()`

Delete all mutation history and make the current state permanent.

Requirements:

- Free all strings in `mutationHistory` except the last one
- Keep only `mutationHistory[mutationCount - 1]` as the new "original"
- Reset `mutationCount = 1`
- Delete `mutation_log.txt` (or clear its contents)
- Create new log with single entry: `0|ORIGINAL_STATE|`

Use case: When you're happy with all changes and want to save memory.

24. `char* undoMultipleMutations(int* indicesToUndo, int count)`

Undo multiple specific mutations at once.

Requirements:

- Receive an array of indices to undo
- Find the **earliest** index in the array
- Return a deep copy of `mutationHistory[earliestIndex - 1]`
- Set `mutationCount = earliestIndex`
- Mark all affected mutations in log file as `|UNDONE`
- If any index is invalid, return `nullptr` and don't undo anything

Example:

- `int indices[] = {3, 5, 7}; // Undo mutations 3, 5, and 7`
- `char* tape = undoMultipleMutations(indices, 3);`
- `// Tape is now at state 2 (before mutation 3)`

Good luck, Archivist. The future depends on your pointer precision.

Little Alex Horne's Recursive Nightmare

[25 marks]

PART A [10 marks]

You are tasked to implement a recursive function that prints a diamond pattern. Helper functions, if needed, must also be recursive.

Prototype

```
void printDiamond(int size, char c1, char c2);
```

The two characters that are passed as arguments are there such that they alternate every level going inwards.

Analyze the example below for size = 4 & 7, c1 = '+' and c2 = '-'.



PART B [15 marks]

The Scenario: You are a contestant on *Taskmaster*. Greg Davies is sitting on his throne, looking down at you with disdain. Little Alex Horne hands you a sealed envelope. You tear it open.

The Task:

"Write a C++ program to generate an X pattern with X's growing from the corners of the X's. The pattern must have the same number of rows and columns, and the number of

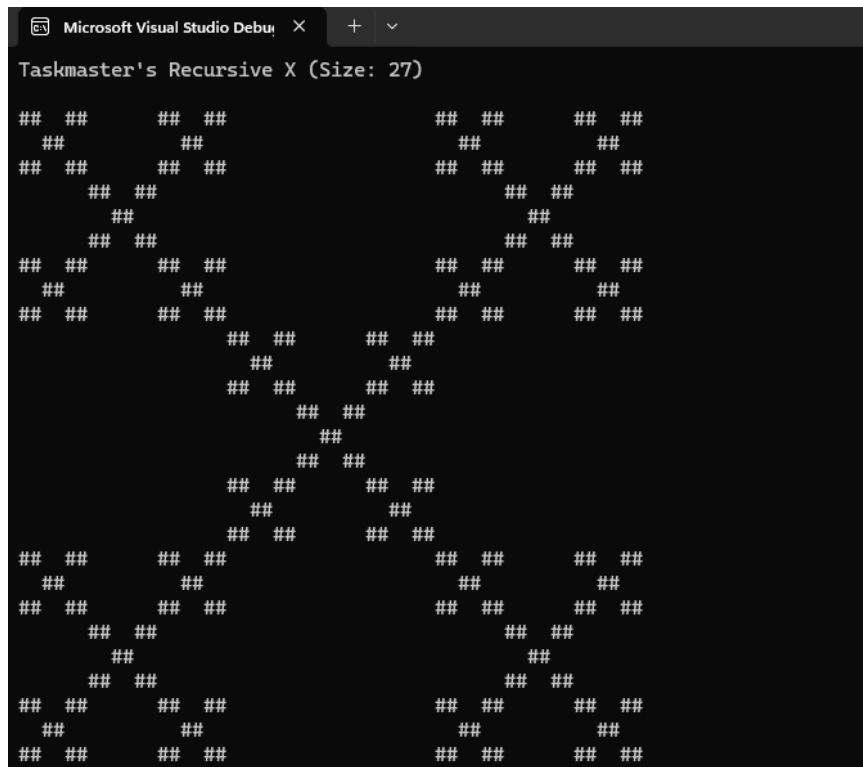
National University of Computer and Emerging Sciences Islamabad Campus

rows and columns must be in power of 3. You may not use any iteration loops (*for*, *while*, *do*). If you use a loop, you will be disqualified and forced to sit on a cake.

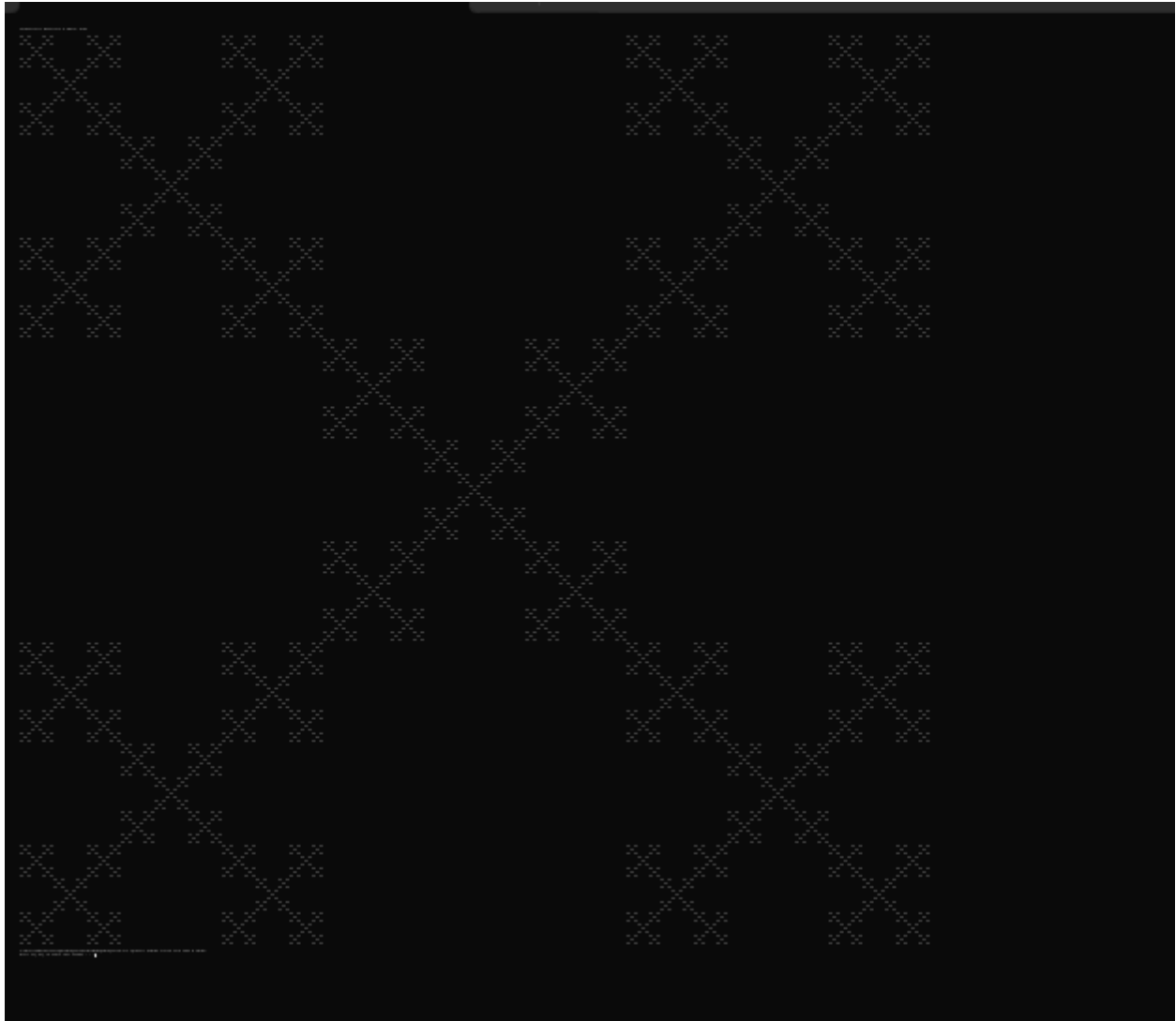
Best recursive logic wins.

Note: If the value given is less than 3, it is taken as 3 and if it's not, it's converted to the last possible value that's a power of 3 (round down). All the helper functions must be recursive as well.

EXAMPLE:

A screenshot of a Microsoft Visual Studio Debug console window. The title bar shows 'Microsoft Visual Studio Debug' and a close button. The console output is titled 'Taskmaster's Recursive X (Size: 27)'. It displays a large 'X' shape composed of '#' characters on a black background. The 'X' is formed by two intersecting diagonal lines of '#' characters. The pattern is symmetric and recursive in nature, with the arms of the 'X' branching outwards. The total size of the pattern is 27x27 characters.

Size = 243



The Ironclad [25 marks]

In a multinational corporation for blockchain and crypto there have been rumors of a whistleblower leaking sensitive information to competitors in exchange for money.

You, an experienced cryptographer, have already developed an encryption system to store each employee's personal information in a .txt file. You are also provided with a file named "file_names.txt" that contains the number and names of the files.

You have to develop an interface and decryption system that works in the following way:

The system asks the user for an 8 character password. No more no less.

National University of Computer and Emerging Sciences

Islamabad Campus

Key Construction

Once the system has the password it extracts the following data from it:

- **KEY (1 byte):** constructed by extracting the LSB's of each character.
- **KEY Modifier (1 byte):** constructed by extracting bit 1 of each character.

Decryption

A txt file containing the names of all the files of the employees will be provided to you for the purpose of loading and decrypting. You'll be performing decryption on each file using the password for that user using the algorithm below:

1. For every file, the encrypted data is going to be 4 times the size of the original data meaning a file containing only 5 characters will have 20 characters once encrypted. (The data is going to spanning a single line only)
2. That encrypted stream is going to be divided into units of size **4 bytes** and each character of the original file is going to be encrypted into a single unit. Each unit will have the following framework:
 - a. **BYTE 0:** The index of the next character in line
 - b. **BYTE 1:** Key modifier for the next character's key
 - c. **BYTE 2:** Garbage (Can be ignored)
 - d. **BYTE 3:** The encrypted character.
3. The contents of the encrypted stream will be shuffled in a spiral manner meaning that you'll always find the starting character at the exact middle of the stream. For the subsequent characters you can use the **BYTE 0** of the unit that tells the index of the next character.
4. The decryption will be done by XORing the encrypted character with the current key and then the current key will be XORed with the key modifier and the result will be the new key for the next character. The key for the very first character is the key constructed from the password.
5. You will be repeating this process until, for a character, the next index is 255. That's when you know that you have decrypted the stream.
6. Display the decrypted stream.

The beauty of this system is that the encrypted files have been made by using the same keys from passwords meaning when a password is entered it successfully decrypts only the correct file. The rest of the files remain garbage no matter what.

Constraints

- No external libraries allowed except **fstream**.
- Static arrays are not allowed.

Mystery Of Severance [25 marks]



Welcome to Macrodata Refinement (MDR)

Refiner: [REDACTED]

Department: MDR

Directive: Protocol "Deep Sonar"

"The work is mysterious and important." — Kier Eagan

You have been retained by Lumon Industries to perform a critical task. The Board requires you to refine raw communication signals into their purest binary form in the form of **Segmented Memory Clusters**.

You will process entire sentences. However, to ensure the integrity of the data, we must treat each word as a volatile memory segment. You will create a **Dynamic Array of Boolean Pointers** to hold these segments.

Furthermore, the Board demands we verify the "weight" of these thoughts. You must interpret these boolean signals as **Short Integers** to calculate a final **Refinement Checksum**.

National University of Computer and Emerging Sciences

Islamabad Campus

And **REMEMBER** these things,

- 1) No contacting your outie (AI), or else, severe consequences can be expected.
- 2) The code must be testable. The Board (GTest) is watching.
- 3) Fail to manage your memory, cause a "Data Hemorrhage" (Memory Leak), and you will be sent to the Break Room.

The Technical Brief

You are required to implement three (3) distinct global functions. These must work in unison to pass the inspection.

1. The Construction

Function Signature: `bool** refineSignal(const char* inputDirective, int& outWordCount, int*& outWordSizes);`

The Directive: You must parse a raw C-string sentence (e.g., "PRAISE KIER"), split it by spaces into **Words**, and convert those words into the **Neural Cluster**.

- **Input:** `inputDirective` (A standard C-string).
- **Outputs (By Reference):**
 - `outWordCount`: You must set this to the number of words found.
 - `outWordSizes`: You must allocate a new `int` array on the heap. You will store the size (in shorts, after padding) of each word's array here. You must point `outWordSizes` to this new array.
- **Return:** You must return the `bool**` structure (The array of pointers).

The Logic (Step-by-Step):

1. **Tokenization:**
 - Parse the `char*`. You must identify individual words separated by spaces.
 - *Note:* "GO AWAY" becomes two distinct segments: "GO" and "AWAY". The space character itself is **discarded**. It serves only as a separator.
2. **The Kier-Approved Morse Translation:**
 - You must convert the characters of a word into a contiguous stream of booleans.
 - **The Dot (.)**: Represented as **1 unit of true**.
 - **The Dash (-)**: Represented as **3 units of true** (i.e., `true, true, true`).
 - **The Inter-Symbol Gap**: Between parts of the *same* letter (e.g., between the dot and dash of 'A'), insert **1 unit of false**.
 - **The Inter-Letter Gap**: Between *different* letters in the same word (e.g., between 'H' and 'I'), insert **2 units of false**.
 - *Visual Example:* "HI" would be `[H-Booleans] + [false, false] + [I-Booleans]`.

National University of Computer and Emerging Sciences

Islamabad Campus

3. The Alignment Protocol (Padding):

- Once a word is fully converted, calculate the total size in bytes (booleans).
- **Constraint:** The Board processes data in `short int` chunks (typically 2 bytes).
- If `(TotalBytes % sizeof(short)) != 0`, you must extend the array with `false` values until it is aligned.
- *Meaning:* If you have 9 booleans, add 1 `false` to make it 10. 10 is divisible by 2.

4. Memory Handoff:

- Store the pointer to this word's array in your `bool**` cluster.
- Store the size (in shorts) in your `outWordSizes` array.
- **Do NOT delete the memory here.** The Board needs it alive for Phase 2.

(Detailed examples of this translation logic are provided in the Protocol Example section below.)

2. The Checksum (The "Bitwise Cascade")

Function Signature: `long long computeChecksum(bool** cluster, int* sizes, int wordCount, int index);`

The Directive: The Board requires a "Rolling Hash" that verifies the integrity of every single bit. Simple addition allows errors to hide. Bitwise Volatility exposes them.

The Logic: You must recursively traverse the words. For each word, you will perform a Non-Linear Accumulation.

1. The Reinterpret Cast:

- Treat the current `bool*` array (`cluster[index]`) as an `unsigned short*`.
- *Note:* We use `unsigned` now. Sign bits are for the weak.

2. The "Cascade" Loop (Inner Volatility):

- You must iterate through the `unsigned shorts` of the current word.
- Constraint 1: You are FORBIDDEN from using an integer index (e.g., `i`) or array subscripting (`[]`) to access data. You must calculate an `end_ptr` based on `sizes[index]` and iterate until your current pointer equals the end pointer.
- Constraint 2 (The Logic): Initialize a local `long long currentVal = 0`.
 - For each `unsigned short (*ptr)`:
 - If `(*ptr)` is EVEN:
 - Left-Shift the `currentVal` by 1.
 - ADD the `(*ptr)` to it.
 - `currentVal = (currentVal << 1) + (*ptr)`
 - If `(*ptr)` is ODD:
 - Right-Shift the `currentVal` by 1.
 - XOR the `(*ptr)` to it.
 - `currentVal = (currentVal >> 1) ^ (*ptr)`

National University of Computer and Emerging Sciences

Islamabad Campus

3. The Recursive Merge:

- You cannot simply add the results. The merge depends on the Word Index.
- If `index` is divisible by 3:
 - Return `currentVal + computeChecksum(index + 1)`.
- Else:
 - Return `currentVal - computeChecksum(index + 1)`.

4. Base Case: If `index == wordCount`, return 0.

3. The Erasure (Manual Cleanup)

Function Signature: `void destroyCluster(bool** cluster, int* sizes, int wordCount);`

The Directive: After the Board (GTest) has verified the data, you must manually hunt down every byte you allocated and free it.

- Delete every `bool*` row.
- Delete the `bool**` spine.
- Delete the `int*` sizes array.

Example Protocol: Refining "GO WORK"

Segment 1: "GO" (Word Index 0 - EVEN)

Composition: *G + Gap + O*

Logic	Raw Boolean Stream	Size (Bytes)	Alignment Check	Final Array (bool)	Shorts (Reinterpreted)
G (--)	1,1,1,0,1,1,1,0,1	9			
Letter Gap	0,0	+2 = 11			

National University of Computer and Emerging Sciences Islamabad Campus

0 (---)	1,1,1,0,1,1,1,0,1 ,1,1	+11 = 22	22 % 2 == 0	[NO PADDING]	11 shorts
----------------	---------------------------	-------------	----------------	-----------------	-----------

Segment 2: "WORK" (Word Index 1 - ODD)

Composition: W + O + R + K (Logic Omitted for Brevity - Assume Total Size is 41 bytes)

Logic	Size (Bytes)	Alignment Check	Action	Final Array (bool)
WORK	41	41 % 2 != 0	PAD +1 False	42 bytes (21 shorts)

The Recursive Math Trace

`computeChecksum` is called with index 0.

1. Index 0 ("GO"):

- **Logic:** The "Bitwise Cascade" (shifts and XORs) on the boolean data produced a `currentVal` of 486.
- **Merge Condition:** `Index (0) % 3 == 0`.
- **Operation:** ADD the result of the recursion.
- **Stack:** `486 + computeChecksum(cluster, sizes, 2, 1)`

2. Index 1 ("WORK"):

- **Logic:** Assume the "Bitwise Cascade" logic on "WORK" produced a `currentVal` of 900.
- **Merge Condition:** `Index (1) % 3 != 0`.
- **Operation:** SUBTRACT the result of the recursion.
- **Stack:** `900 - computeChecksum(cluster, sizes, 2, 2)`

3. Index 2 (Base Case):

National University of Computer and Emerging Sciences

Islamabad Campus

- **Logic:** `index == wordCount`.
- **Result:** Returns 0.

4. The Unwind (Final Calculation):

- **Step 2 Resolves:** $900 - 0 = 900$
- **Step 1 Resolves:** $486 + 900 = 1386$
- **Final Output:** 1386

Security Notice:

- **Strings:** Usage of `std::string`, `std::vector`, or `std::stringstream` is immediate grounds for termination. Use `<cstring>` functions (`strlen`, `strcpy`, `strtok` etc.) only.
- **Leaks:** We will call `refineSignal` then `destroyCluster` 10,000 times. If RAM usage creeps up, you fail.

Praise Kier.

Get to work. The quarter is ending soon.

The Void Walkers [50 marks]

``Winter is coming``

TO: Alan Turing, Bletchley Park

FROM: MI6 Intelligence directorate

DATE: January 23, 1944

SUBJECT: Enigma's Big Brother

We believed breaking the enigma was the end. We were wrong.

We have intercepted a new, high-density digital signal originating from the german high command. They call it **VOID**. The machine used to decode it is called a **VOID WALKER**.

Several of our spies have gotten their hands on the blueprint of the so-called "VOID WALKER". But the problem at hand is that none is capable of implementing that blueprint in the language of the future: **C++**.

We have several new recruits who claim to be masters of C++ and will try and construct the VOID WALKER.

National University of Computer and Emerging Sciences

Islamabad Campus

Details

The data packets being received are arrays of type **unsigned char** of dynamic sizes and no real pattern. It seems like a complete noise of bytes to the untrained eye.

Your job is to use the blueprint and construct the VOID WALKER in hopes of decoding VOID and winning the war.

The Blueprint

The stream consists of multiple packets. Every packet in the stream begins with a standard 5-byte header:

- Byte 0: identifies the type of the packet (1, 2, or 3).
- Bytes 1-4: An integer representing the total size of the packet (header + payload) in bytes.
- Sentinel: The stream ends when byte with data 255 is reached.

CRITICAL:

- Endianness: The machine uses **Little Endian** Architecture. Which means that the data is Least significant Byte to the Most significant Byte. E.g. FF 00 00 00 is going to be read as 00 00 00 FF which is 255 in integer.

The Packet Types

Packet 1: The Necrotic Chain

The enemy transmits the strengths of their troops to one another in the form of a flattened array. But the array is of no primitive type rather it consists of a certain number of units (specified in the header) where each unit consists of:

- Bytes 0-3: Unit ID (int)
- Bytes 4-7: Power (float)
- Bytes 8-11: Master ID (int) - The ID of their senior officer

The Mission

Implement a recursive function that takes in the following inputs:

- A pointer to the stream
- A target ID

LOGIC: Recursively sum the "Power" of the target Commander PLUS the power of every subordinate (and their subordinates) in the chain. (No loops, helpers, if made, must be recursive)

OUTPUT: output the calculated power like this: **"Necrotic Power: 20"**

National University of Computer and Emerging Sciences

Islamabad Campus

Packet 2: The Frostbite Labyrinth

The enemy also transmits one another the safe routes to follow in order to avoid the trap minefields. The packet consists of a flattened array representing a square grid. In that array:

- 'S': Start Point
- 'E': End Point
- '#': Mine
- '.': Path

The Mission

Implement a recursive function to find a path from 'S' to 'E'. (No loops, helpers if needed, must be recursive)

OUTPUT: output "Frostbite Labyrinth: SOLVABLE" or "Frostbite Labyrinth: UNSOLVABLE". And if solvable you also have to output the solved maze.

Packet 3: THE SCREAM

The enemy has managed a way to also send audio messages encrypted via the VOID and decrypted via the VOID WALKERS.

But the encryption is primitive as it just adds noise to the audio.

The packet consists of the following payload in case of packet 3:

- **HEADER:** 8 bytes: Sample Rate(int) + Channel Count(int)
- **PAYLOAD:** A sequence of raw floats between -1.0 and 1.0

The Mission

Implement a smooth function that iterates through the payload and replaces every sample with the average of itself with its neighbors.

The number of channels tells you the stride length in pointer arithmetic. E.g. For channel length 2 the payload will be: L R L R L R...

So the neighbour of L is always 1 step over.

Constraints

- No external libraries allowed.
- The use of '[' operator not allowed
- Static arrays are not allowed
- All the allocation must be precise meaning no space must be wasted

Make Christopher proud.

Good Luck

Key Reminder of what pointers are like:

